433 MHz RF Communication with Arduino Practice

Arduino Code: Basic Transmission

Simple TX (No Library)

```
#define TX_PIN 12

void setup() {
  pinMode(TX_PIN, OUTPUT);
  Serial.begin(9600);
}
```

```
void sendBit(bool bit) {
  if (bit) {
    // Send 1: Long pulse
    digitalWrite(TX_PIN, HIGH);
    delayMicroseconds(600);
    digitalWrite(TX_PIN, LOW);
    delayMicroseconds(200);
  } else {
    // Send 0: Short pulse
    digitalWrite(TX_PIN, HIGH);
    delayMicroseconds(200);
    digitalWrite(TX_PIN, LOW);
    delayMicroseconds(600);
  }
}
```

```
void sendByte(byte data) {
  for (int i = 7; i >= 0; i--) {
    bool bit = (data >> i) & 1;
    sendBit(bit);
  }
}

void sendPacket(byte command) {
  // Preamble (wake receiver)
  for (int i = 0; i < 8; i++) {
    sendByte(0xAA);
  }

  // Sync byte
  sendByte(0xF0);
  // Data
  sendByte(command);
  // Simple checksum
  sendByte(~command);
}
```

```
void loop() {
  if (Serial.available()) {
    char cmd = Serial.read();

    if (cmd == '1') {
      sendPacket(0x01);
      Serial.println("Sent: Button 1");
    }
  }

  delay(100);
}
```

Arduino Code: Using RCSwitch Library

TX with RCSwitch (Easier!)

```
#include <RCSwitch.h>

RCSwitch mySwitch = RCSwitch();

void setup() {
  Serial.begin(9600);

  // Enable transmitter on pin 12
  mySwitch.enableTransmit(12);

  // Optional: set pulse length (default 350)
  mySwitch.setPulseLength(350);

  // Optional: set repeat (default 10)
  mySwitch.setRepeatTransmit(10);
```

```
void loop() {
  if (Serial.available()) {
    char cmd = Serial.read();

    switch (cmd) {
      case '1':
        mySwitch.send(123456, 24);  // Send 24-bit code
        Serial.println("Sent: Code 123456");
        break;

      case '2':
        mySwitch.send(789012, 24);
        Serial.println("Sent: Code 789012");
        break;

      case '3':
        // Send binary string
        mySwitch.send("000000000001010101010101");
        Serial.println("Sent: Binary code");
        break;

      case '4':
        // Send tristate (for DIP switch devices)
        mySwitch.sendTriState("0F0F0F0F");
        Serial.println("Sent: Tristate code");
        break;
    }
  }

  delay(100);
}
```

Arduino Code: RCSwitch Receiver

RX with RCSwitch

```
#include <RCSwitch.h>

RCSwitch mySwitch = RCSwitch();

void setup() {
  Serial.begin(9600);

  // Enable receiver on interrupt 0 (pin 2)
  mySwitch.enableReceive(0);  // Use digitalPinToInterrupt(2) on newer Arduino

  Serial.println("433MHz Receiver Ready");
  Serial.println("Waiting for signals...");
}
```

```
void loop() {
  if (mySwitch.available()) {

    // Get received value
    unsigned long value = mySwitch.getReceivedValue();

    if (value == 0) {
      Serial.println("Unknown encoding");
    } else {
      Serial.print("Received: ");
      Serial.print(value);
      Serial.print(" / ");
      Serial.print(mySwitch.getReceivedBitlength());
      Serial.print(" bits ");
      Serial.print("Protocol: ");
      Serial.println(mySwitch.getReceivedProtocol());

      // Take action based on code
      if (value == 123456) {
        Serial.println("-> Button 1 pressed!");
        digitalWrite(LED_BUILTIN, HIGH);
      } else if (value == 789012) {
        Serial.println("-> Button 2 pressed!");
        digitalWrite(LED_BUILTIN, LOW);
      }
    }

    mySwitch.resetAvailable();
  }
}
```

Arduino Code: Remote Switch Clone

Learn and Replay Remote Codes

```
#include <RCSwitch.h>

RCSwitch mySwitch = RCSwitch();

#define MAX_CODES 10
unsigned long learnedCodes[MAX_CODES];
int learnedBits[MAX_CODES];
int learnedProtocol[MAX_CODES];
int codeCount = 0;
```

```
void setup() {
  Serial.begin(9600);
  mySwitch.enableReceive(0);   // RX on pin 2
  mySwitch.enableTransmit(12); // TX on pin 12

  Serial.println("RF Remote Cloner");
  Serial.println("Commands:");
  Serial.println("L – Learn new code");
  Serial.println("1–9 – Replay learned code");
  Serial.println("S – Show all codes");
}
```

```
void loop() {
  // Check for serial commands
  if (Serial.available()) {
    char cmd = Serial.read();

    if (cmd == 'L' || cmd == 'l') {
      learnCode();
    } else if (cmd >= '1' && cmd <= '9') {
      int index = cmd - '1';
      if (index < codeCount) {
        replayCode(index);
      } else {
        Serial.println("No code stored at that position");
      }
    } else if (cmd == 'S' || cmd == 's') {
      showCodes();
    }
  }

  // Monitor for incoming codes
  if (mySwitch.available()) {
    Serial.print("Detected: ");
    Serial.print(mySwitch.getReceivedValue());
    Serial.print(" (");
    Serial.print(mySwitch.getReceivedBitlength());
    Serial.println(" bits)");

    mySwitch.resetAvailable();
  }
}
```

```
void learnCode() {
  if (codeCount >= MAX_CODES) {
    Serial.println("Memory full!");
    return;
  }

  Serial.println("Press remote button...");

  while (!mySwitch.available()) {
    delay(10);
  }

  learnedCodes[codeCount] = mySwitch.getReceivedValue();
  learnedBits[codeCount] = mySwitch.getReceivedBitlength();
  learnedProtocol[codeCount] = mySwitch.getReceivedProtocol();

  Serial.print("Learned! Code ");
  Serial.print(codeCount + 1);
  Serial.print(": ");
  Serial.print(learnedCodes[codeCount]);
  Serial.print(" (");
  Serial.print(learnedBits[codeCount]);
  Serial.println(" bits)");

  codeCount++;
  mySwitch.resetAvailable();
}
```

```
void replayCode(int index) {
  Serial.print("Sending code ");
  Serial.print(index + 1);
  Serial.print(": ");
  Serial.println(learnedCodes[index]);

  mySwitch.setProtocol(learnedProtocol[index]);
  mySwitch.send(learnedCodes[index], learnedBits[index]);

  delay(100);
}

void showCodes() {
  Serial.println("\nLearned codes:");
  for (int i = 0; i < codeCount; i++) {
    Serial.print(i + 1);
    Serial.print(": ");
    Serial.print(learnedCodes[i]);
    Serial.print(" (");
    Serial.print(learnedBits[i]);
    Serial.print(" bits, Protocol ");
    Serial.print(learnedProtocol[i]);
    Serial.println(")");
  }
}
```

Security Concerns (CRITICAL!)

⚠️ HUGE PROBLEM: Anyone can intercept and replay!

433MHz Has ZERO Built-in Security!

```
Attack #1: Eavesdropping
  1. Attacker uses 433MHz receiver
  2. Captures your remote code
  3. Knows: "Code 12345 opens garage"
  4. Can monitor ALL your transmissions

Attack #2: Replay Attack
  1. Record your garage door remote code
  2. Replay it later to open door
  3. Works even if you're not home!
  4. No way for receiver to tell difference

Attack #3: Brute Force
  1. Try all possible codes
  2. 24-bit code = 16 million combinations
  3. At 10 codes/second = 18 days
  4. Fixed codes are vulnerable
```

How to Add Security

Rolling Codes (Best Practice)

```
// Simple rolling code example
unsigned long counter = 0;  // Increments with each use

void sendSecure(byte command) {
  // Create packet with counter
  unsigned long packet = ((unsigned long)counter << 8) | command;

  // Send packet
  mySwitch.send(packet, 32);

  // Increment for next time
  counter++;

  // Save counter to EEPROM so it persists
  EEPROM.put(0, counter);
}
```

```
// Receiver side
unsigned long lastCounter = 0;

void receiveSecure() {
  if (mySwitch.available()) {
    unsigned long packet = mySwitch.getReceivedValue();

    // Extract counter and command
    unsigned long rxCounter = packet >> 8;
    byte command = packet & 0xFF;

    // Check if counter is greater than last
    if (rxCounter > lastCounter && rxCounter < lastCounter + 100) {
      // Valid! Accept command
      executeCommand(command);
      lastCounter = rxCounter;
      EEPROM.put(0, lastCounter);
    } else {
      // Reject! Replay attack detected
      Serial.println("Security: Replay rejected");
    }

    mySwitch.resetAvailable();
  }
}
```

Still not perfect, but much better than fixed codes!